SOFTWARE ENGINEERING PROJECT PROPOSAL

Smart Internship Management System

Project Proposal Report

Submitted By

Cheerag — 1024170391
Sachinjeet — 1024170376

B.Tech Computer Science Engineering
Thapar Institute of Engineering and Technology
Patiala, Punjab

Academic Year 2026–27

Table of Contents

1. Abstract

2. Introduction

3. Background and Existing System

4. Problem Statement

5. Objectives

6. Scope of the Project

7. Stakeholder Analysis

8. User Roles and Permissions

9. Functional Requirements

10. Non-Functional Requirements

11. System Modules

12. Proposed System Architecture

13. Database Design

14. Use Case Analysis

15. Detailed Application Workflow

16. Smart Internship Recommendation

17. Data Flow and UML Modeling Plan

18. Software Development Methodology

19. Project Timeline

20. Testing Strategy

21. Technology Stack

22. Security and Privacy Considerations

23. Project Risks and Mitigation

24. Expected Outcomes

25. Limitations

26. Future Enhancements

27. Conclusion

28. References

1. Abstract

The Smart Internship Management System is a centralized software platform proposed to simplify the complete internship lifecycle for students, companies, placement coordinators, and system administrators. In many academic environments, internship opportunities are communicated through multiple channels such as email, messaging groups, spreadsheets, company websites, and external job portals. As a result, students may find it difficult to discover suitable opportunities, verify eligibility, keep track of deadlines, and monitor the status of their applications.

The proposed system provides a unified platform in which students can create professional profiles, maintain resumes, specify skills and preferences, discover internships, apply online, and monitor their applications. Companies can create verified profiles, publish internship opportunities, define eligibility criteria, review applicants, shortlist candidates, schedule interviews, and update final selection outcomes. Placement or faculty coordinators can verify companies and opportunities, monitor student applications, and generate institutional reports.

A major component is a smart internship recommendation module. The initial implementation uses an explainable weighted matching mechanism based on skill overlap, eligibility, preferred domain, and location. This provides useful personalization without requiring a large machine-learning dataset and can later be extended into a machine-learning recommendation engine.

2. Introduction

Internships play an important role in the academic and professional development of university students. They provide exposure to real-world development environments, professional communication, teamwork, project management, and industry-specific technologies. For employers, internship programs provide an opportunity to identify potential talent and evaluate students before making longer-term hiring decisions.

Despite their importance, internship management is often fragmented. A student may receive one opportunity through a college group, another through email, and another through an external platform. Application deadlines, eligibility requirements, stipend information, location, work mode, and required skills may be presented in different formats. After applying, students often maintain their own spreadsheets to remember which opportunities are pending, shortlisted, rejected, or completed.

Recruiters must publish opportunities, specify eligibility criteria, collect applications, review candidates, coordinate interviews, and record final outcomes. Placement cells may additionally verify organizations, communicate opportunities, track student participation, and prepare reports. A centralized system can reduce repetitive work and provide a consistent source of information.

The Smart Internship Management System is proposed as a centralized platform for managing these activities. It is designed around different user roles, a structured database, a defined application workflow, and a recommendation module that helps students identify relevant opportunities.

• Motivation: reduce fragmented information, manual tracking, and lack of personalization.

• Need: create one structured source of internship information and application history.

• Value: improve transparency for students while reducing administrative workload.

3. Background and Existing System

The current internship ecosystem usually combines college communication channels, external platforms, company career pages, online forms, and manually maintained spreadsheets. Each approach solves part of the problem, but there is usually no single workflow connecting opportunity discovery, eligibility checking, application, selection, and institutional tracking.

For example, a company may publish an internship through an external portal while the college separately circulates the same opportunity through a messaging group. Students may submit an application through a form and later receive an email about an interview. The placement cell may maintain another spreadsheet to track selected students. This creates multiple copies of information and increases the possibility of inconsistent or outdated records.

The proposed platform does not attempt to replace every external internship portal. Instead, it creates a controlled academic ecosystem where verified internship opportunities, student profiles, applications, and institutional monitoring can be managed together.

• Fragmented information across multiple sources.

• Manual tracking of application status.

• Inconsistent internship formats.

• Limited personalization.

• High administrative workload.

• Difficulty producing reliable reports.

4. Problem Statement

Students, recruiters, and placement coordinators face difficulties in managing internships because opportunity discovery, application submission, recruitment communication, and record keeping are often distributed across multiple systems. This fragmentation can cause missed deadlines, duplicate work, unclear application status, inconsistent records, and difficulty identifying suitable opportunities.

The problem is to design and develop a centralized internship management system that supports the complete internship workflow, provides role-specific functionality, maintains reliable records, and intelligently recommends relevant opportunities to students.

The system should be modular enough to allow future integration with advanced recommendation, notification, analytics, and external recruitment services.

5. Objectives

The primary objective is to develop a centralized, secure, and user-friendly platform for managing internships.

• Provide secure authentication and role-based access control.

• Allow students to maintain academic and professional profiles.

• Provide resume and skill management.

• Allow companies to register and create structured internship listings.

• Support verification of companies and internship opportunities.

• Provide internship search and filtering.

• Check eligibility before or during applications.

• Provide online applications and status tracking.

• Allow recruiters to manage applicants and selection stages.

• Provide dashboards and institutional reports.

• Implement a smart recommendation mechanism.

• Maintain a reliable history of applications and outcomes.

6. Scope of the Project

The project focuses on the internship management lifecycle within a university or college ecosystem.

Student scope: registration, profile management, skills, preferences, resume, internship search, applications, status tracking, and recommendations.

Company scope: company registration, verification information, internship creation, eligibility criteria, applicant review, shortlisting, interviews, and selection outcomes.

Coordinator scope: organization and internship verification, application monitoring, and institutional reporting.

Administrator scope: user management, role management, moderation, and system-level configuration.

Out of scope for the initial prototype: real monetary transactions, legally binding employment contracts, and direct integration with every external internship portal.

7. Stakeholder Analysis

Students are the primary consumers of internship opportunities and require a simple way to discover, compare, and apply to suitable positions. Companies need efficient applicant management and structured recruitment workflows. Coordinators require institutional oversight and accurate records. Administrators maintain platform integrity.

Stakeholder	Main Interest	System Need
Students	Find and apply	Search, recommendations, applications, tracking
Companies	Recruit interns	Posting, applicants, shortlisting, interviews
Coordinators	Monitor activity	Verification, tracking, reports
Administrators	Maintain platform	Users, roles, moderation
Institution	Improve process	Reliable records, statistics, transparency

8. User Roles and Permissions

Student: access to profile management, internship discovery, applications, recommendations, and application history.

Recruiter: access to company profile, internship listings, applicant management, shortlisting, interviews, and selection.

Placement Coordinator: access to company/listing verification, application monitoring, and institutional statistics.

Administrator: access to user management, roles, reported content, and system-level configuration.

9. Functional Requirements

The system shall support the following functional requirements:

- New user registration and secure login.
- Role recognition and role-specific authorization.
- Student profile creation and editing.
- Student skills, preferences, and resume management.
- Company profile creation and verification submission.
- Creation and management of internship listings.
- Storage of eligibility criteria and required skills.
- Internship search and filtering.
- Eligibility checking.
- Online internship application.
- Duplicate application prevention.
- Application status tracking.
- Recruiter applicant viewing.
- Shortlisting and rejection.
- Interview scheduling.
- Final selection status.
- Company and internship verification.
- Internship recommendations.
- Application history.
- Dashboards and reports.
- Optional deadline and status notifications.

10. Non-Functional Requirements

Security: Authentication and authorization must prevent unauthorized access. Passwords should be securely hashed and protected APIs must validate identity and permissions.

Usability: The interface should use clear navigation, readable forms, understandable status labels, and consistent actions.

Performance: Login, search, profile loading, and application submission should respond efficiently under expected project load.

Reliability: Application and internship records must remain consistent, with validation before critical database changes.

Scalability: The architecture should support growth in users and listings without major redesign.

Maintainability: Modules should have clear responsibilities and interfaces to simplify debugging and extension.

Availability: The system should remain accessible during major application periods.

Privacy: Student resumes, contact information, and application records must only be accessible to authorized users.

11. System Modules

Authentication and Authorization: manages registration, login, logout, password management, and permissions.

Student Profile: stores academic information, skills, interests, preferences, and resume information.

Company: manages organization details and verification status.

Internship Management: handles creation, editing, publishing, closing, searching, and filtering of listings.

Application Management: connects students with internships and maintains status history.

Recommendation: evaluates compatibility between students and available internships.

Verification: supports coordinator review of companies and internship listings.

Dashboard and Reporting: summarizes application counts, internship activity, and selection statistics.

Administration: manages users, roles, reports, and platform configuration.

12. Proposed System Architecture

The system will follow a layered client-server architecture. The separation of presentation, API, business logic, and database responsibilities makes the system easier to develop, test, and maintain.

Presentation Layer: provides interfaces for students, recruiters, coordinators, and administrators.

API Layer: exposes REST endpoints and validates requests.

Business Logic Layer: implements eligibility, application status transitions, recommendations, verification, and authorization.

Data Access Layer: performs database operations and isolates persistence logic.

Database Layer: stores users, companies, internships, applications, skills, interviews, and notifications.

High-level flow: User Interface → REST API → Authentication/Business Logic → Database.

When a student applies, the frontend sends the request to the API. The backend authenticates the student, verifies that the internship is active, checks eligibility and duplicate application rules, stores the application, and returns the updated status.

13. Database Design

The proposed relational database contains multiple connected entities.

User: user ID, name, email, password hash, role, and account status.

StudentProfile: student ID, academic details, branch, graduation information, preferences, and resume reference.

Company: company ID, organization name, description, website, and verification status.

Internship: internship ID, company ID, title, description, required skills, duration, stipend, location, work mode, deadline, and status.

Application: application ID, student ID, internship ID, submission date, status, and remarks.

Skill: standardized skill identifier and skill name.

StudentSkill / InternshipSkill: relationship entities connecting skills to students and internships.

Interview: interview ID, application ID, schedule, mode, and outcome.

Notification: recipient, message, timestamp, and read status.

A student can apply to multiple internships, while an internship can receive many applications. The Application entity therefore resolves the many-to-many relationship. Primary keys, foreign keys, unique constraints, and validation rules will be used to maintain database integrity.

14. Use Case Analysis

Student use cases: register/login, manage profile, add skills, upload resume, browse internships, search/filter, view details, apply, track status, and view recommendations.

Recruiter use cases: register company, submit verification, create/edit/close internships, view applicants, shortlist, reject, schedule interviews, and update outcomes.

Coordinator use cases: review company information, verify companies, review listings, monitor applications, view statistics, and generate reports.

Administrator use cases: manage users, roles, listings, reports, and platform activity.

15. Detailed Application Workflow

The internship lifecycle begins when a recruiter creates an internship listing containing title, description, eligibility criteria, required skills, duration, location, stipend, work mode, and deadline.

If institutional verification is enabled, the listing is reviewed by a coordinator. Once approved, the internship becomes visible to eligible students.

A student views the listing and submits an application using stored profile information and a resume. The backend validates identity, internship status, eligibility, and duplicate application rules. A new application is created with an initial Applied status.

The recruiter reviews applications. Candidates can be moved to Shortlisted, after which an interview may be scheduled. Following the recruitment process, the candidate can be marked Selected or Rejected.

Each status transition can be timestamped. This provides transparency and creates a complete application history that can later support reporting and analytics.

16. Smart Internship Recommendation

The recommendation component is the distinguishing feature of the system. It helps students discover opportunities that match their profile instead of relying only on manual searches.

Possible inputs include technical skills, soft skills, academic branch, academic eligibility, preferred domain, preferred location, work-mode preference, and internship duration preference.

A first version can calculate a weighted score:

RecommendationScore = 0.50(SkillMatch) + 0.25(Eligibility) + 0.15(DomainMatch) + 0.10(LocationMatch)

Skill matching can be calculated as the number of matched required skills divided by the total number of required skills. Eligibility can be binary or partial depending on the criteria. Domain and location can be normalized to comparable scores.

For example, a student with Python, SQL, Git, and data-analysis skills who prefers software/data roles will receive a high score for an internship requiring Python and SQL in that domain, provided academic eligibility is satisfied.

The rule-based approach is transparent, easy to test, and practical for an academic prototype. After enough historical data is available, it can be upgraded to machine learning.

17. Data Flow and UML Modeling Plan

The final project documentation will contain formal software engineering models.

Context-Level DFD: the system is represented as one major process interacting with Student, Recruiter, Coordinator, and Administrator entities.

Level-1 DFD: the system can be divided into Authentication, Profile Management, Internship Management, Application Management, Recommendation, Verification, and Reporting processes.

Use Case Diagram: shows which actors interact with which system features.

Class Diagram: represents major classes such as User, Student, Company, Internship, Application, Interview, Skill, and Notification and their relationships.

Sequence Diagrams: can document application submission and recruiter shortlisting.

Activity Diagram: can model the complete internship application lifecycle.

ER Diagram: represents database entities, keys, and relationships.

18. Software Development Methodology

The project will follow an Agile-oriented iterative methodology. This allows the team to build the core system first and progressively add smart and administrative features.

Iteration 1 — Requirements: identify stakeholders, functional and non-functional requirements, constraints, and scope.

Iteration 2 — Design: architecture, database schema, UML, DFD, wireframes, and API specifications.

Iteration 3 — Core Development: authentication, profiles, companies, internships, and applications.

Iteration 4 — Smart Features: recommendation scoring, eligibility matching, dashboards, and notifications.

Iteration 5 — Testing: unit, integration, system, usability, security, and acceptance testing.

Iteration 6 — Deployment and Documentation: final integration, GitHub repository, report, presentation, and user documentation.

19. Project Timeline

The following schedule is a proposed academic implementation plan.

Phase	Activities	Deliverables
Week 1	Requirements and research	Problem definition, SRS draft
Week 2	Architecture and modeling	UML, DFD, ER design
Week 3	Database and authentication	Schema, login module
Week 4	Student/company modules	Profiles and company management
Week 5	Internship/application modules	Listings and application workflow
Week 6	Recommendations/dashboard	Smart matching and analytics
Week 7	Testing and debugging	Test cases and defect log
Week 8	Final integration	Prototype, report, PPT

20. Testing Strategy

Unit Testing: individual functions such as login validation, eligibility calculation, recommendation scoring, and status transitions will be tested independently.

Integration Testing: frontend, REST APIs, and database communication will be tested together.

System Testing: complete workflows will be tested from the perspective of each role.

Security Testing: unauthorized access, invalid tokens, incorrect permissions, and malicious inputs will be tested.

Usability Testing: a small group of users can perform common tasks and provide feedback.

ID	Test Case	Expected Result
TC01	Valid student login	Student dashboard opens.
TC02	Invalid password	Error message; login fails.
TC03	Create student profile	Profile stored successfully.
TC04	Create internship	Listing stored with correct status.
TC05	Search by skill	Matching internships displayed.
TC06	Eligible application	Application created with Applied status.
TC07	Duplicate application	Second application rejected.
TC08	Expired internship	Application not accepted.
TC09	Shortlist applicant	Status changes to Shortlisted.
TC10	Final selection	Student sees Selected status.
TC11	Unauthorized admin request	Access denied.
TC12	Recommendation request	Ranked internships displayed.

21. Technology Stack

Frontend: Flutter or a web frontend depending on the final implementation choice.

Backend: Node.js with Express for REST APIs and business logic.

Database: MySQL for relational storage.

API: REST API architecture.

Authentication: token-based authentication with role-based authorization.

Version Control: Git and GitHub.

API Testing: Postman or an equivalent REST client.

The separation between frontend, backend, and database supports modular development and makes it possible to replace one layer without redesigning the entire system.

22. Security and Privacy Considerations

The system will store potentially sensitive information such as student contact information, resumes, academic records, and application history. Security and privacy must therefore be considered from the beginning of the design.

Important controls include secure password hashing, authentication tokens with suitable expiry, role-based authorization, server-side validation, protection against unauthorized record access, controlled resume access, safe database queries, secure uploaded-file handling, and logging of important administrative operations.

The principle of least privilege should be applied so that each role receives only the permissions necessary for its responsibilities. For example, a student should not be able to view another student's private application information, and a recruiter should only access applicants belonging to their organization's internships.

23. Project Risks and Mitigation

Scope creep can result in an unfinished project. The team should freeze an MVP containing the essential workflow before implementing optional features.

Authentication defects can create serious security problems. Established libraries and tested authorization middleware should be used.

Database errors can cause inconsistent application records. Constraints, validation, and transactions should be applied where appropriate.

The recommendation system could consume excessive development time if machine learning is attempted too early. The initial version should use explainable rule-based scoring.

Integration problems between team members' modules can be reduced by defining API contracts early and testing endpoints continuously.

Time constraints should be handled by implementing core modules first and optional enhancements later.

24. Expected Outcomes

The completed project is expected to provide a functional internship management application with secure multi-role authentication, student profile and resume management, company and recruiter management, structured internship listings, search and filtering, online applications, application status tracking, recruiter shortlisting, company/internship verification, smart recommendations, dashboards, a relational database, REST APIs, documented test cases, and a GitHub repository containing source code and documentation.

The prototype should demonstrate the complete internship lifecycle and provide a clear foundation for future expansion.

25. Limitations

The initial prototype may have limitations. Recommendation quality depends on the quality of student profiles and internship descriptions. A rule-based recommendation system may not capture complex preferences as effectively as a trained machine-learning model.

Real-world company verification may require institutional or legal processes that are outside the scope of the prototype. External job-board integration and real payment processing are also outside the initial implementation.

These limitations are acceptable for an academic prototype and provide clear directions for future development.

26. Future Enhancements

Future versions can introduce AI-based resume parsing, automatic extraction of skills and education, machine-learning recommendation models, email and push notifications, calendar integration, advanced placement analytics, recruiter analytics, interview automation, and integration with approved external internship sources.

A skill-gap analysis feature could compare a student's current skills with desired internships and recommend learning resources or skills that would improve eligibility.

Advanced analytics could identify popular skills, application conversion rates, company participation, internship domains, and student placement trends.

27. Conclusion

The Smart Internship Management System is proposed as a centralized software solution for managing the internship lifecycle in an academic environment. It addresses fragmented opportunity discovery, manual application tracking, inconsistent records, and lack of personalized recommendations.

The system supports students, companies, placement coordinators, and administrators through role-specific modules. Students receive a structured way to discover and apply for internships, recruiters receive tools to manage applicants, coordinators receive verification and monitoring capabilities, and administrators receive system-level control.

The recommendation module adds a meaningful smart feature while keeping the first implementation practical through an explainable weighted scoring model. The project also provides substantial scope for requirements analysis, UML modeling, DFD design, database development, modular architecture, testing, risk management, Git-based collaboration, and iterative development.

The project can therefore serve both as a useful academic prototype and as a foundation for a more comprehensive internship and placement platform.

28. References

1. Ian Sommerville, *Software Engineering*, Pearson.

2. Roger S. Pressman and Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.

3. MySQL Documentation.

4. Node.js Documentation.

5. Express.js Documentation.

6. Flutter Documentation.

7. GitHub Documentation.